

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №5: разработка микросервисов

Выполнил:

Кузнецов Артур

Группа К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2025 г.

Задача

- выделить самостоятельные модули в вашем приложении;
- провести разделение своего API на микросервисы (минимум, их должно быть 3);
- настроить сетевое взаимодействие между микросервисами.

Ход работы

1 Выбор функций монолита и разделение на микросервисы

На первом этапе была проанализирована структура исходного монолитного приложения. Внутри него были выделены три основные функциональные области:

- управление пользователями, их регистрацией, аутентификацией и ролями;
- работа с рецептами, их ингредиентами, типами блюд, уровнями сложности и шагами приготовления;
- социальные взаимодействия вокруг рецептов: лайки, комментарии, сохранённые рецепты и подписки между пользователями.

Эти области были выбраны в качестве естественных границ будущих микросервисов. Такое разбиение позволило минимизировать связность между компонентами и сделать каждую часть системы автономной.

При разделении монолита основное внимание уделялось контроллерам, моделям и связям между сущностями. Исходно все домены (пользователи, рецепты и социальные функции) находились в одном приложении и пользовались общей базой данных. В процессе перехода к микросервисной архитектуре были выполнены следующие шаги:

- функциональность регистрации, входа в систему, работы с пользователями и ролями была сгруппирована и перенесена в отдельный сервис auth-service;

- операции, связанные с кулинарным контентом (создание и просмотр рецептов, управление ингредиентами, типами блюд, уровнями сложности и шагами), были выделены в `recipe-service`;

- логика лайков, комментариев, сохранённых рецептов и подписок была сконцентрирована в `social-service`.

При этом связи между сущностями были пересмотрены. Например, в сервисе рецептов модель `Recipe` хранит только числовой идентификатор автора, а не прямую ORM-ссылку на сущность пользователя из другого сервиса. Такой же подход реализован в социальном сервисе для ссылок на пользователей и рецепты. Это позволило отказаться от прямых внешних ключей между базами разных сервисов и перевести взаимодействие доменов на уровень API и идентификаторов.

Общая логика, которая ранее дублировалась в разных частях монолита (проверка JWT-токена, проверка прав доступа и обработка ошибок), была вынесена в выделенный общий модуль `common-service`. Он используется всеми микросервисами как общая библиотека и обеспечивает единообразное поведение по аутентификации и обработке ошибок.

2 Структура и ответственность сервисов

2.1 Auth-service

`auth-service` отвечает за:

- регистрацию и аутентификацию пользователей;
- выпуск JWT-токенов;
- управление сущностями пользователя и роли;
- базовые проверки корректности вводимых данных.

В контроллере `authController` реализованы операции регистрации и входа. При регистрации пароль хэшируется, создаётся запись пользователя в собственной базе данных сервиса и генерируется JWT-токен с информацией о пользователе и его роли. При входе производится проверка

существования пользователя и сверка пароля, после чего выдаётся новый JWT-токен.

Сервис поднимает отдельный экземпляр express-приложения (порт 3000), регистрирует собственные роутеры (/auth, /user, /role), использует подключённую базу PostgreSQL и отдельную Swagger-документацию на /docs.

2.2 Recipe-service

recipe-service инкапсулирует работу с кулинарным доменом:

- сущности, связанные с рецептом и его вариациями;
- операции CRUD над рецептами и связанными данными;
- инициализация справочников (базовый набор типов блюд, уровней сложности, ингредиентов).

Модель Recipe содержит только идентификатор автора, а не ссылку на сущность пользователя из auth-service, что подчёркивает независимость баз данных разных микросервисов. При запуске сервис инициализирует собственную базу и выполняет начальное заполнение справочников с помощью сервисов `initDishTypeService`, `initRecipeDifficultyService`, `initIngredientService`.

Сервис работает на порту 3001, имеет собственные роутеры (/recipe, /recipe-step, /ingredient, /dish-type, /recipe-difficulty, /recipe-ingredient) и отдельную Swagger-документацию на /docs.

2.3 Social-service

- social-service агрегирует весь функционал, связанный с социальными взаимодействиями:
- комментарии к рецептам;
- лайки;
- сохранённые рецепты;
- подписки одного пользователя на другого.

Внутри сервиса определены модели для каждой из этих сущностей и контроллеры, реализующие соответствующие эндпоинты `/comment`, `/like`, `/saved-recipe`, `/subscription`. Для ссылок на пользователей и рецепты используются числовые идентификаторы, а не внешние ключи в другие базы, что позволяет гарантировать автономность базы данных `social-service`.

Сервис запускается на порту 3002, использует собственный экземпляр базы данных и отдельную Swagger-документацию на `/docs`.

2.4 Common-service

Для уменьшения дублирования кода и унификации поведения был выделен общий сервис-библиотека `common-service`. Его основное назначение – предоставить повторно используемые `middleware` и утилиты для всех микросервисов. В его состав входят:

- `authMiddleware` – единая реализация проверки JWT-токена в заголовке;
- `adminOnlyMiddleware` – проверка наличия аутентифицированного пользователя с ролью администратора и возврат статуса 403 при отсутствии прав;
- `errorHandler` – общий обработчик ошибок `express`, логирующий стек и возвращающий клиенту стандартизированный ответ с сообщением об ошибке и кодом 500;
- дополнительные утилиты и типы, которые позволяют единообразно реализовать повторяющиеся проверки и вспомогательные функции в разных сервисах.

Таким образом, `common-service` обеспечивает общую инфраструктурную логику, а бизнес-логика остаётся внутри специализированных микросервисов.

3 Работа с Docker

Для трёх основных микросервисов был настроен собственный `Dockerfile`. Конфигурации унифицированы и включают:

- использование лёгкого базового образа `node:22-alpine` для уменьшения размера контейнеров;
- установку рабочей директории `WORKDIR /app`;
- копирование служебных файлов конкретного сервиса (`package*.json`, `tsconfig.json`, `.env`) в контейнер;
- копирование каталога `common-service` внутрь образа, чтобы микросервисы могли использовать общий код как локальную зависимость;
- выполнение `npm install` для установки зависимостей;
- копирование исходников (`src`) конкретного сервиса и сборка TypeScript-кода командой `npm run build`;

Такая структура `Dockerfile` позволяет собирать изолированные образы для каждого микросервиса, при этом повторно использовать общий код из `common-service`.

Для совместного запуска всех компонентов был настроен общий файл `docker-compose.yml` в корне проекта. В нём описаны:

- три сервисных контейнера:
 - `auth-service` (порт 3000),
 - `recipe-service` (порт 3001),
 - `social-service` (порт 3002);
- три отдельных контейнера с базами PostgreSQL:
 - `auth-db` для `auth-service`,
 - `recipe-db` для `recipe-service`,
 - `social-db` для `social-service`;
- общий пользовательский мостовой сетевой сегмент `app-network`, через который все сервисы и базы данных взаимодействуют друг с другом по внутренним DNS-имёнам контейнеров.

Каждый микросервис собирается из своего `Dockerfile` в директории `microservices`, получает собственный набор переменных окружения из

соответствующего .env-файла и зависит от своей базы данных. Для PostgreSQL настроены именованные Docker-тома (auth-db-data, recipe-db-data, social-db-data), которые сохраняют данные между перезапусками контейнеров.

Сетевое взаимодействие между микросервисами реализовано следующим образом:

- все контейнеры подключены к общей сети app-network, что позволяет обращаться к другим сервисам по их логическим именам (например, из одного микросервиса к другому по имени сервиса из docker-compose.yml);

- каждый сервис имеет собственный внешний порт, проброшенный на хостовую машину (3000, 3001, 3002), что позволяет фронтенду и разработчику обращаться к ним по HTTP;

- взаимодействие между микросервисами строится через HTTP-запросы с использованием единообразной аутентификации на основе JWT-токенов: токены выдаёт auth-service, а проверка их корректности в остальных сервисах выполняется через общее authMiddleware из common-service.

Таким образом, docker-compose.yml задаёт единое окружение для запуска трёх микросервисов и трёх независимых баз данных, а также конфигурирует общую сеть, в рамках которой сервисы могут обмениваться данными по HTTP.

Вывод

В ходе работы монолитное приложение было последовательно разделено на три специализированных микросервиса: `auth-service` (пользователи и аутентификация), `recipe-service` (рецепты и кулинарный домен) и `social-service` (социальные взаимодействия). Для каждого сервиса была выделена собственная база данных PostgreSQL и переработаны модели таким образом, чтобы устранить прямые ORM-ссылки между доменами и оставить только идентификаторы сущностей. Общая инфраструктурная логика (проверка JWT-токенов, авторизация и обработка ошибок) была вынесена в модуль `common-service`, который переиспользуется всеми микросервисами. Для развёртывания настроены отдельные Dockerfile для каждого сервиса и общий `docker-compose.yml`, обеспечивающий запуск контейнеров, конфигурацию сетевого взаимодействия и изолированных хранилищ данных, что позволило на практике отработать принципы микросервисной архитектуры и организации сетевого взаимодействия между сервисами.